

UNIVERSIDAD DE ESPECIALIDADES ESPÍRITU SANTO
FACULTAD DE INGENIERÍA

CINESPOT

Análisis, planificación y diseño de la solución

Sistemas Operativos

JOSIAS PEREZ, JORGE COLAMARCO

29 de octubre de 2026

Descripción y alcance de CineSpot

CineSpot es un proyecto académico que se desarrollará en Python para simular una plataforma de consulta y reserva de asientos en funciones de cine. Permitirá consultar la disponibilidad de los asientos y solicitar la reserva de uno libre para una función determinada. Antes de confirmar cada solicitud, el sistema deberá validar el asiento y actualizar su estado. El problema principal consiste en mantener una disponibilidad confiable cuando varias solicitudes coinciden, de modo que un asiento no pueda asignarse más de una vez para la misma función.

Este documento reúne el análisis de modelos de comunicación y la planificación del diseño de la solución. En la fase descrita se ha elegido memoria compartida con hilos y se propone un diccionario para representar los asientos. La comprobación de disponibilidad y su actualización se protegerán mediante threading.Lock. Los fragmentos de código incluidos son ejemplos de diseño; no constituyen evidencia de implementación ejecutada, pruebas realizadas ni cambios publicados en el repositorio.

Análisis de modelos de comunicación

Para analizar la coordinación, se considera que distintas solicitudes pueden atenderse al mismo tiempo. Cada solicitud debe comprobar el estado del asiento elegido y, si sigue disponible, reservarlo. Si dos solicitudes coinciden sobre el mismo asiento, ambas podrían leerlo como libre antes de que alguna registre el cambio. Por eso, la comprobación y la actualización deben coordinarse como una sola operación.

Unidades de ejecución y recurso compartido

Cada unidad de ejecución representa un trabajador que procesa la solicitud de un usuario. El recurso compartido es el mapa de asientos de una función: registra cada asiento y su estado, disponible o reservado. El identificador de la función forma parte de la identidad del registro, porque un mismo asiento puede estar disponible en una función y reservado en otra. Los trabajadores deben cooperar para consultar y cambiar ese estado sin duplicar asignaciones.

Comparación de modelos

En memoria compartida, varios hilos acceden a datos comunes y los accesos concurrentes que pueden entrar en conflicto requieren sincronización (OpenMP Architecture Review Board, 2021). En el paso de mensajes, los procesos intercambian solicitudes y respuestas explícitamente (Message Passing Interface Forum, 2023). La tabla 1 compara su aplicación al control de asientos de CineSpot.

Tabla 1. Comparación de los modelos de comunicación en CineSpot

Modelo	Aplicación en CineSpot	Ventajas y límites
Memoria compartida	Varios hilos atienden solicitudes y consultan el mismo mapa de asientos. Un threading.Lock protege la sección que verifica que el asiento esté disponible y lo cambia a reservado (Python Software Foundation, s. f.-c).	Es directo para una simulación en un solo proceso y evita intercambiar mensajes por cada solicitud. Sin el bloqueo alrededor de la comprobación y el cambio, dos hilos podrían reservar el mismo asiento.
Paso de mensajes	Cada proceso trabajador envía función, asiento y usuario por una multiprocessing.Queue a un coordinador. Este es dueño del mapa, valida y actualiza el estado y devuelve una respuesta (Python Software Foundation, s. f.-b).	El mapa tiene un único responsable y los trabajadores no escriben directamente en él. Requiere gestionar mensajes y respuestas; el coordinador puede convertirse en un punto de espera.

Conclusión del análisis

Para el alcance académico actual se recomienda memoria compartida con hilos. La simulación mantendrá el inventario de cada función en una estructura común dentro de la aplicación; un bloqueo breve alrededor de la comprobación y la reserva protegerá la operación evitando duplicados. Este diseño requiere menos coordinación adicional que una cola y un proceso administrador. El paso de mensajes sería conveniente si CineSpot separara sus trabajadores en procesos o servicios independientes. La recomendación responde al alcance descrito y no representa una ventaja de rendimiento medida.

Planificación y diseño de la solución

A partir del análisis anterior, se definen las estructuras de datos, las responsabilidades de las clases y las reglas de acceso concurrente necesarias para desarrollar la simulación en Python. Se conserva el modelo de memoria compartida con hilos; el diseño todavía deberá implementarse y validarse.

Recurso compartido por función

El recurso compartido será el mapa de asientos de cada función. Su identidad lógica será el par (id_funcion, id_asiento): A1 en F01 será un recurso distinto de A1 en F02. Cada registro indicará si está disponible o reservado y, en la representación compleja, qué usuario realizó la reserva. Todos los trabajadores de una función deberán acceder al mismo mapa y al mismo bloqueo.

La regla de consistencia será que cada par función–asiento admite como máximo una reserva confirmada. Un asiento disponible tendrá usuario igual a None; uno reservado tendrá un identificador de usuario válido. Una función o un asiento inexistente deberá producir un rechazo sin modificar el inventario. Las funciones y sus asientos se crearán antes de iniciar los hilos.

Representaciones propuestas del recurso

Tipo básico con un entero

Para un asiento de una función se propone un int: 0 significará disponible y 1 reservado. Por ejemplo, estado_F01_A1 = 0 representará A1 libre en F01. La reserva requerirá comprobar el valor y cambiarlo a 1 dentro del mismo bloqueo. El entero expresa la ocupación de un solo asiento; para modelar la sala completa se necesitan varias variables o una estructura que las agrupe. Un contador de cupos por sí solo no identifica qué asiento se reservó.

Tipo complejo con un diccionario

Se propone agrupar registros por función y por asiento. Las claves identificarán cada recurso y los valores contendrán estado y usuario. El siguiente ejemplo es ilustrativo; F01, F02 y U01 no corresponden a datos reales del proyecto.

```
funciones = {
    "F01": {"A1": {"estado": 0, "usuario": None}},
    "F02": {"A1": {"estado": 1, "usuario": "U01"}}
}
```

El diccionario permite asociar claves con valores y modificar sus registros (Python Software Foundation, s. f.-a). El estado y el usuario se actualizarán juntos bajo el bloqueo de la función para mantener su correspondencia.

Comparación de las representaciones

La tabla 2 compara las propuestas según la información que necesita CineSpot. Ambas requieren sincronizar la secuencia de comprobar y reservar; el tipo de dato no sustituye el control del acceso concurrente.

Tabla 2. Comparación de las representaciones del recurso

Criterio	Entero por asiento	Diccionario por función y asiento
Información	Registra 0 o 1 para un asiento.	Registra estado y usuario para cada asiento.
Identificación	Depende del nombre de la variable o de un contexto adicional.	Las claves distinguen función y asiento.
Extensión	Agregar usuarios u otros datos exige estructuras adicionales.	Admite campos adicionales en cada registro.
Complejidad	Es sencillo para explicar una ocupación aislada.	Exige definir claves, validar registros y proteger las actualizaciones.
Sincronización	Debe proteger la lectura y el cambio de 0 a 1.	Debe proteger la comprobación y la actualización conjunta del registro.

Elección para la implementación prevista

Se utilizará el diccionario porque el problema exige identificar asientos de funciones distintas y asociar cada reserva con un usuario. El entero se conservará como representación básica del estado dentro del registro. La elección responde a las necesidades de información; no supone una ventaja de rendimiento medida. El esquema anidado representa el inventario lógico, mientras que en el diseño de clases cada SalaCine encapsulará el mapa interno de su función.

Diseño de las clases de Python

Se proponen tres clases. SalaCine encapsulará un mapa por función; TrabajadorReserva atenderá una solicitud como hilo; y SimuladorCineSpot organizará la ejecución. Esta distribución separa las solicitudes del control del recurso, de modo que los trabajadores no modificarán los registros directamente.

SalaCine

Responsabilidad: administrar los asientos de una función y proteger sus accesos. Atributos propuestos: id_funcion, _asientos y _lock. El primero identificará la función; _asientos contendrá el diccionario de sus asientos; _lock será una única instancia de threading.Lock creada al inicializar el objeto.

consultar_asiento(id_asiento) validará la existencia del asiento y devolverá una copia de su registro bajo el mismo bloqueo. reservar_asiento(id_asiento, usuario) comprobará la existencia y disponibilidad, y actualizará estado y usuario en una sola sección crítica. Devolverá un resultado que distinga reserva confirmada, asiento ocupado y solicitud inválida. Una consulta previa será informativa: la decisión final se tomará de nuevo al reservar.

TrabajadorReserva

Responsabilidad: representar una solicitud concurrente. Se propone que herede de threading.Thread. Sus atributos serán sala, id_asiento, usuario y resultado. sala será una referencia al objeto SalaCine de la función solicitada; los otros datos pertenecerán a cada solicitud individual.

run() invocará sala.reservar_asiento() y guardará el resultado en el propio trabajador. El hilo no tendrá un mapa ni un Lock independientes. Para atender solicitudes sobre la misma función, varios trabajadores compartirán exactamente la misma referencia a SalaCine.

SimuladorCineSpot

Responsabilidad: inicializar las funciones y coordinar los hilos. Sus atributos serán funciones, un diccionario que relaciona cada identificador con su objeto SalaCine; solicitudes, con los datos de entrada; y trabajadores, con los hilos creados.

ejecutar() validará los identificadores de función, creará un trabajador por solicitud válida y llamará a start() para iniciarlos. Después recorrerá los trabajadores con join() y reunirá sus resultados. No reservará asientos directamente ni mantendrá el bloqueo mientras espera la terminación de los hilos.

Relación entre los hilos y el recurso

El hilo principal creará una SalaCine por función antes de iniciar los trabajadores. Dos solicitudes a F01 compartirán su mapa y su bloqueo; una solicitud a F02 usará otro objeto. Así, el bloqueo de F01 no impedirá acceder a F02. La espera con join() permitirá leer cada resultado una vez terminado el trabajador; start() y join() pertenecen a la interfaz de Thread (Python Software Foundation, s. f.-c).

Sección crítica y sincronización

La sección crítica estará en reservar_asiento(). Comprenderá la consulta del registro, la comprobación de disponibilidad y la sustitución del registro por su estado reservado con el usuario correspondiente. Estas operaciones deberán ser indivisibles frente a los demás hilos que accedan a la misma función.

Problema sin coordinación

Si dos trabajadores solicitan A1 de F01, podría producirse el entrelazado de la tabla 3. El estado final podría conservar solo al último usuario, aunque ambos trabajadores hubieran confirmado la reserva.

Tabla 3. Ejemplo de condición de carrera sin bloqueo

Paso	Trabajador 1	Trabajador 2
1	Lee A1 como disponible.	
2		Lee A1 como disponible.
3	Registra U01 y confirma.	
4		Registra U02 y confirma.

Uso propuesto de threading.Lock

Se emplea un Lock por objeto SalaCine y, por tanto, por función. Todos los métodos que consulten o modifiquen el mapa usarán ese mismo bloqueo. Mientras un trabajador lo mantenga adquirido, los demás deberán esperar antes de entrar. La sentencia with adquirirá el bloqueo al entrar y lo liberará al salir del bloque, incluso si ocurre una excepción (Python Software Foundation, s. f.-c).

El siguiente fragmento ilustra el método previsto. Se asume que la solicitud ya contiene un usuario válido y que self._lock y self._asientos se inicializarán en SalaCine.

```
def reservar_asiento(self, id_asiento, usuario):
    with self._lock:
        registro = self._asientos.get(id_asiento)
        if registro is None:
            return "solicitud_invalida"
        if registro["estado"] == 1:
            return "asiento_ocupado"
        self._asientos[id_asiento] = {
            "estado": 1, "usuario": usuario
        }
        return "reserva_confirmada"
```

El segundo trabajador volverá a comprobar el registro después de adquirir el bloqueo y rechazará la reserva si el primero ya lo ocupó. Las consultas devolverán copias para evitar escrituras externas sin protección. Los mensajes, las esperas simuladas y la presentación de resultados se realizarán fuera del bloqueo. El método de reserva no llamará a otro método que intente adquirir nuevamente el mismo Lock.

Conclusión del diseño

El diseño organizará el inventario por función y encapsulará cada mapa en SalaCine. Sus trabajadores actualizarán estado y usuario dentro de la misma sección crítica. Así se establece la coordinación que deberá implementarse para evitar reservas duplicadas. En esta etapa, las decisiones se han fundamentado a nivel de análisis y planificación; la implementación y sus pruebas deberán documentarse cuando estén disponibles en el proyecto.

Registro de actualizaciones del documento

Este registro permite mantener una única versión acumulativa de CineSpot. Cada avance de clase deberá incorporarse en las secciones correspondientes de este mismo archivo, sin generar documentos académicos independientes. Las fechas registran avances documentales y no acreditan por sí solas commits en el repositorio.

Tabla 4. Historial de actualización del contenido

Fecha	Etapa	Contenido incorporado
13/10/2026	Análisis	Descripción del problema, concurrencia, comparación de memoria compartida y paso de mensajes, y selección del modelo.
29/10/2026	Planificación y diseño	Estructuras de datos, reglas de consistencia, clases propuestas, sección crítica y uso previsto de threading.Lock.

Bibliografía

Message Passing Interface Forum. (2023). MPI: A message-passing interface standard version 4.1. https://www.mpi-forum.org/docs/mpi-4.1/mpi41-report/mpi41-report.htm

OpenMP Architecture Review Board. (2021). OpenMP application programming interface specification version 5.2. https://www.openmp.org/spec-html/5.2/openmp.html

Python Software Foundation. (s. f.-a). Built-in types. Python 3.14 documentation. Recuperado el 29 de octubre de 2026 de https://docs.python.org/3.14/library/stdtypes.html#mapping-types-dict

Python Software Foundation. (s. f.-b). multiprocessing - Process-based parallelism. Python 3.14 documentation. Recuperado el 13 de octubre de 2026 de https://docs.python.org/3.14/library/multiprocessing.html

Python Software Foundation. (s. f.-c). threading - Thread-based parallelism. Python 3.14 documentation. Recuperado el 29 de octubre de 2026 de https://docs.python.org/3.14/library/threading.html